

DOKUMEN PENGGUNAAN AI CODING TOOL

PENGEMBANGAN SISTEM PERPUSTAKAAN BERBASIS WEB

ARSITEKTUR 2 SERVICE / MICROSERVICE

1. IDENTITAS DAN INFORMASI PROYEK

Nama Proyek	Sistem Perpustakaan Berbasis Web
Jenis Pengembangan	Pengembangan menjadi 2 service / microservice
Service 1	Book Service
Service 2	Borrowing Service
AI Coding Tool	Goggle Gemini & ChatGPT

2. LATAR BELAKANG PENGGUNAAN AI CODING TOOL

AI coding yang digunakan sebagai alat bantu dalam pengembangan proyek sistem perpustakaan berbasis web, yang mana disini kami mengubah yang awalnya dalam satu struktur web yang menggabungkan berbagai fungsi sistem dalam satu folder. Pada tahap pengembangan selanjutnya, arsitektur sistem dikembangkan menjadi microservice dengan memisahkan fungsi utama sistem menjadi dua service, yaitu Book Service dan Borrowing Service. Pemisahan ini dilakukan berdasarkan tanggung jawab masing-masing fungsi agar setiap service memiliki fungsi yang lebih spesifik dan dapat dikembangkan secara terpisah.

3. PEMBAGIAN 2 SERVICE PADA PROYEK

Proyek perpustakaan dikembangkan menjadi dua service. Pembagian dilakukan agar setiap service memiliki tanggung jawab yang jelas.

No.	Service	Fungsi Utama	Komunikasi
1	Service peminjaman	Mengelola informasi/data buku	API ke service buku

		yang sudah dipinjam dan menyediakan API data buku.	
2	Service Pengembalian	Mengelola proses/data penngembalian	API

5. TAHAPAN PENGGUNAAN AI CODING TOOL

No.	Tahap	Tujuan	Prompt yang Digunakan	Hasil/Tindakan
1	Review dan perencanaan Arsitektur	Mengevaluasi proyek perpustakaan yang sudah ada dan menentukan pembagian tanggung jawab service	Saya sedang melanjutkan proyek sistem peminjaman buku perpustakaan yang sudah saya kembangkan sebagian menjadi arsitektur microservice. Saya butuh bantuan kamu sebagai AI coding Tool untuk mereview, memperbaiki dan melengkapi kode yang sudah ada – bukan membuat dari nol	AI memberikan panduan Langkah-langkah pengerjaan, memberikan Struktur folder proyek microservice. Kemudian mulsi pengerjaan yakni tahap pertama AI memberikan service 1: Book Service. Service 2: Borrowing Service
2	Pembuatan Service Buku	Service ini khusus untuk mengelola	Ubah sistem peminjaman buku	Menampilkan semua data buku sebelumnya

		katalog buku dan status ketersediaan buku.	yang sebelumnya menggunakan JavaScript localStorage menjadi arsitektur microservice. Buat Book Service menggunakan Node.js dan Express untuk mengelola data buku dan status ketersediaan buku.	kemudian, membuat serve/API service book.
3	Pembuatan Service Peminjaman	Membuat server/API Service Peminjaman	Buat Borrowing Service menggunakan Node.js dan Express. Service harus memvalidasi maksimal 3 buku aktif per mahasiswa dan masa peminjaman 7 hari.”	Kode diperiksa dan disesuaikan dengan kebutuhan proyek.
4	Komunikasi API	Menghubungkan kedua service	Implementasikan komunikasi API antara Borrowing Service dan Book Service untuk mengecek ketersediaan buku	Endpoint dan alur komunikasi diuji.

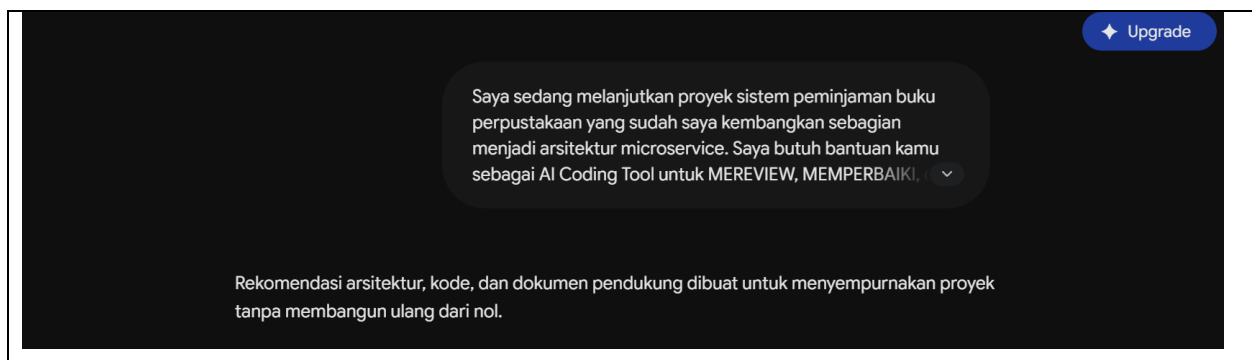
			sebelum transaksi peminjaman.	
5	Debugging	Memperbaiki error	Kode saya menghasilkan error berikut: [tempel error]. Apa penyebab dan cara memperbaikinya?	Solusi AI diuji dan hanya perbaikan yang sesuai yang diterapkan.

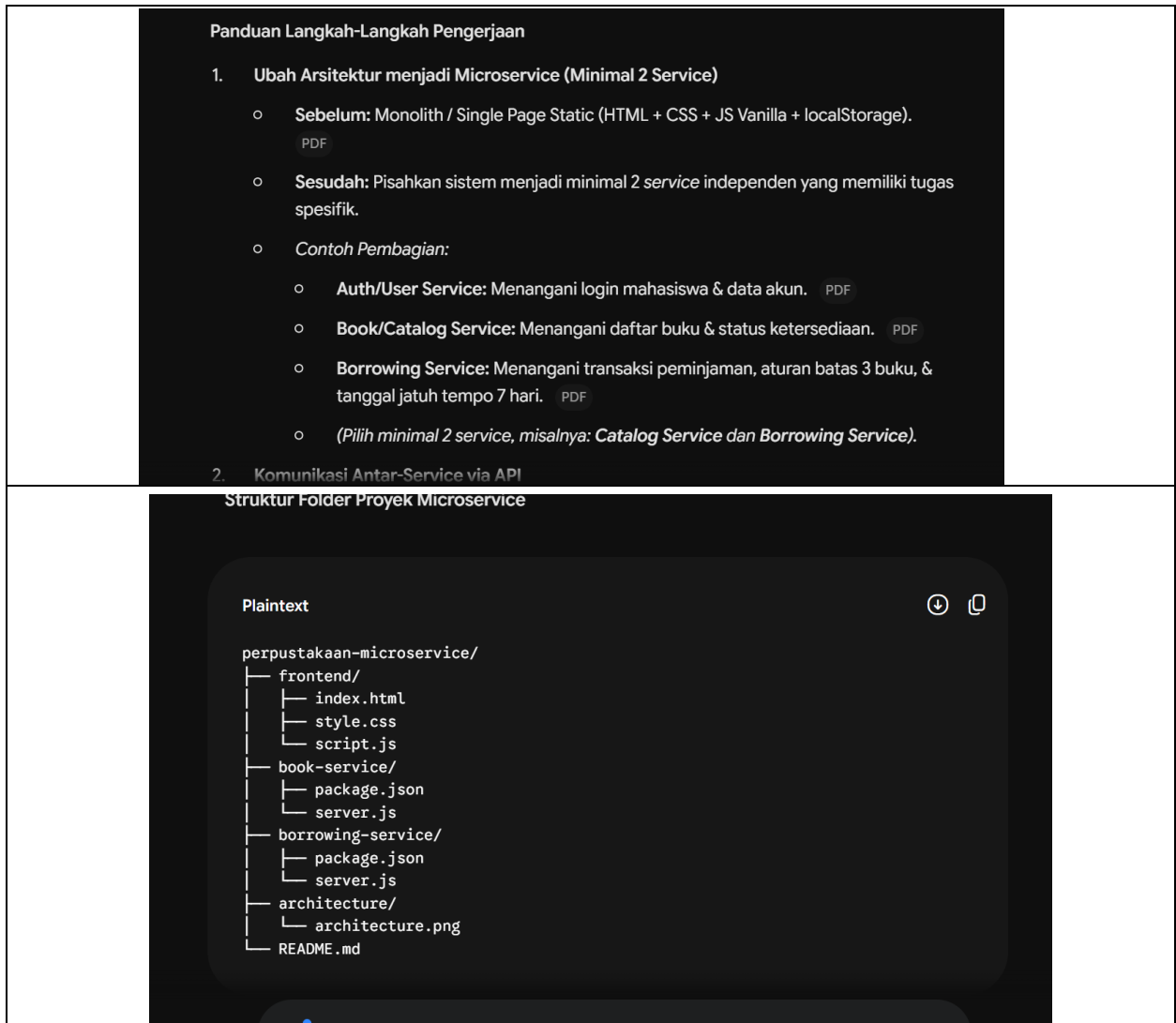
No	Bagian Yang Diperiksa	Masalah/Hasil Pemeriksaan	Tindakan Perbaikan
1.	Port Server	URL dan Port <i>borrowing-service</i> dipanggil langsung secara <i>hardcode</i> (http://localhost:5001), serta belum ada penanganan port dinamis dari <i>environment variable</i> .	Mengubah konfigurasi port agar menggunakan <code>process.env.PORT</code> dengan <i>fallback</i> port default (Book Service: 5001 , Borrowing Service: 5002 , dan Auth Service: 5003). Menjadikan URL komunikasi dinamis melalui <code>process.env.BOOK_SERVICE_URL</code> .
2.	Endpoint API	1. Belum ada endpoint untuk mekanisme login/autentikasi mahasiswa (BR-01 / AC-01). 2. Pengecekan ketersediaan buku dan pembaruan status buku dilakukan secara terpisah, berisiko <i>race</i>	1. Membuat service otentikasi baru (auth-service) dengan endpoint POST <code>/api/auth/login</code> dan POST <code>/api/auth/verify</code> . 2. Membuat endpoint atomic PUT <code>/api/books/:id/borrow</code> pada <i>book-service</i> untuk langsung mengecek dan mengunci status buku. 3. Menambahkan endpoint POST <code>/api/borrowings/:id/return</code> dan PUT

		<i>condition (AC-03).</i> 3. Belum ada endpoint pengembalian buku (<i>return</i>).	/api/books/:id/return untuk fitur pengembalian buku.
3.	Komunikasi antar-service	1. <i>Borrowing-service</i> menerima nim langsung dari <i>request body</i> tanpa memvalidasi identitas ke service mana pun. 2. Tidak ada <i>error handling</i> jika <i>book-service</i> sedang mati/tidak merespons.	1. Menambahkan middleware <code>authenticateToken</code> pada <i>borrowing-service</i> untuk memverifikasi token ke <i>auth-service</i> via HTTP REST API sebelum memproses peminjaman. 2. Menambahkan blok try-catch dan penanganan status HTTP 503 Service Unavailable saat komunikasi antar-service gagal.

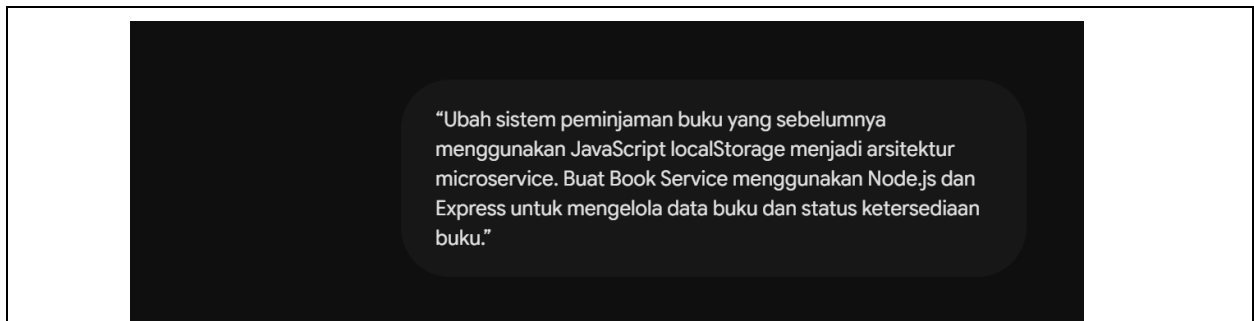
8. BUKTI SCREENSHOT PENGGUNAAN AI

Screenshot 1. Percakapan AI saat merancang arsitektur 2 service.





Screenshot 2. Percakapan AI saat membuat Book service dan Borrowing Service.



1. Isi Kode book-service

- o Masuk ke folder `book-service` , lalu buat file baru bernama `server.js` . JS+1
- o Paste kode berikut ke dalamnya:

JavaScript

```
const express = require('express');
const cors = require('cors');
const app = express();
app.use(cors());
app.use(express.json());

let books = [
  { id: "BK01", judul: "Coding Basics", pengarang: "Andrea Hirata", gambar: "images/gambar1.jpg"},
  { id: "BK02", judul: "Computer Science", pengarang: "Pramoedya Ananta Toer", gambar: "images/gambar2.jpg"},
  { id: "BK03", judul: "Linux Karnael", pengarang: "Thierry Gayet", gambar: "images/gambar3.jpg"},
  { id: "BK04", judul: "Code Firts", pengarang: "Oreilly", gambar: "images/gambar4.jpg"}
];
```

JavaScript

```
// GET: Ambil semua buku
app.get('/api/books', (req, res) => {
  res.json(books);
});

// GET: Cek detail & ketersediaan buku tertentu
app.get('/api/books/:id', (req, res) => {
  const book = books.find(b => b.id === req.params.id);
  if (!book) return res.status(404).json({ message: "Buku tidak ditemukan" });
  res.json(book);
});

// PUT: Update status ketersediaan buku (dipanggil oleh Borrowing Service)
app.put('/api/books/:id/status', (req, res) => {
  const { isAvailable } = req.body;
  const book = books.find(b => b.id === req.params.id);
  if (!book) return res.status(404).json({ message: "Buku tidak ditemukan" });

  book.isAvailable = isAvailable;
  res.json({ message: "Status buku berhasil diperbarui", book });
});
```

2. Service 2: Borrowing Service (borrowing-service/server.js)

Service ini mengurus aturan bisnis peminjaman (maksimal 3 buku, jatuh tempo 7 hari) dan berkomunikasi dengan Book Service menggunakan `fetch` / `axios` .

JavaScript

```
const express = require('express');
const cors = require('cors');
const app = express();
app.use(cors());
app.use(express.json());

let loans = [];

// Helper: Hitung tanggal jatuh tempo 7 hari
```

JavaScript

```
// Helper: Hitung tanggal jatuh tempo 7 hari
function calculateDueDate() {
  const date = new Date();
  date.setDate(date.getDate() + 7);
  return date.toISOString().split('T')[0];
}

// GET: Ambil peminjaman berdasarkan NIM
app.get('/api/borrowings/:nim', (req, res) => {
  const userLoans = loans.filter(l => l.nim === req.params.nim && l.status === 'AKTIF');
  res.json(userLoans);
});

// POST: Proses Transaksi Peminjaman (Melibatkan Komunikasi API ke Book Service)
app.post('/api/borrowings', async (req, res) => {
  const { nim, bookId } = req.body;

  // 1. Validasi Aturan Bisnis: Batas Maksimal 3 Buku
  const activeLoans = loans.filter(l => l.nim === nim && l.status === 'AKTIF');
  if (activeLoans.length >= 3) {
```

```
const bookResponse = await fetch(`http://localhost:5001/api/books/${bookId}`);
if (!bookResponse.ok) {
  return res.status(404).json({ message: "Buku tidak ditemukan pada katalog" });
}
const book = await bookResponse.json();

// Validasi Ketersediaan Buku
if (!book.isAvailable) {
  return res.status(400).json({ message: "Buku sedang tidak tersedia untuk dipinjam" });
}

// 3. Buat Transaksi Peminjaman Baru
const newLoan = {
  id: `LN-${nim}-${Date.now()}`,
  nim,
  bookId,
  tanggalPinjam: new Date().toISOString().split('T')[0],
  tanggalJatuhTempo: calculateDueDate(),
  status: 'AKTIF'
};
```

```
};
loans.push(newLoan);

// 4. Update Status Buku di Book Service menjadi Tidak Tersedia
await fetch(`http://localhost:5001/api/books/${bookId}/status`, {
  method: 'PUT',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ isAvailable: false })
});

res.status(201).json({ message: "Peminjaman berhasil", loan: newLoan });
} catch (error) {
  res.status(500).json({ message: "Gagal terhubung ke Book Service", error: error.message });
}
};

app.listen(5002, () => console.log('Borrowing Service berjalan di port 5002'));
```

Screenshot 4. Percakapan AI saat membuat/menjelaskan komunikasi API.

1. Update File `frontend/script.js`

Buka file `frontend/script.js` kamu, lalu sesuaikan bagian yang bertugas mengambil data buku dan melakukan peminjaman agar memanggil API backend:

- Untuk Mengambil Data Buku (Ganti `fetch/data` lokal lama dengan API Book Service):

```
JavaScript

// Ambil data buku dari Book Service (Port 5001)
async function fetchBooks() {
  try {
    const response = await fetch('http://localhost:5001/api/books');
    const books = await response.json();
    renderBooks(books); // Panggil fungsi render UI kamu
  } catch (error) {
    console.error('Gagal mengambil data buku:', error);
  }
}
```

+ Minta Gemini Flash

1. Komunikasi Frontend ke `book-service` (Port 5001)

- Alur API: `frontend/script.js` mengirim permintaan `GET` ke `http://localhost:5001/api/books` untuk mengambil daftar katalog buku.
- Error: Muncul pesan kesalahan *CORS Policy* di *console browser* (`Failed to fetch`), sehingga daftar buku tidak bisa tampil.
- Perbaikan: Menambahkan *middleware* `CORS` di `book-service/server.js`:

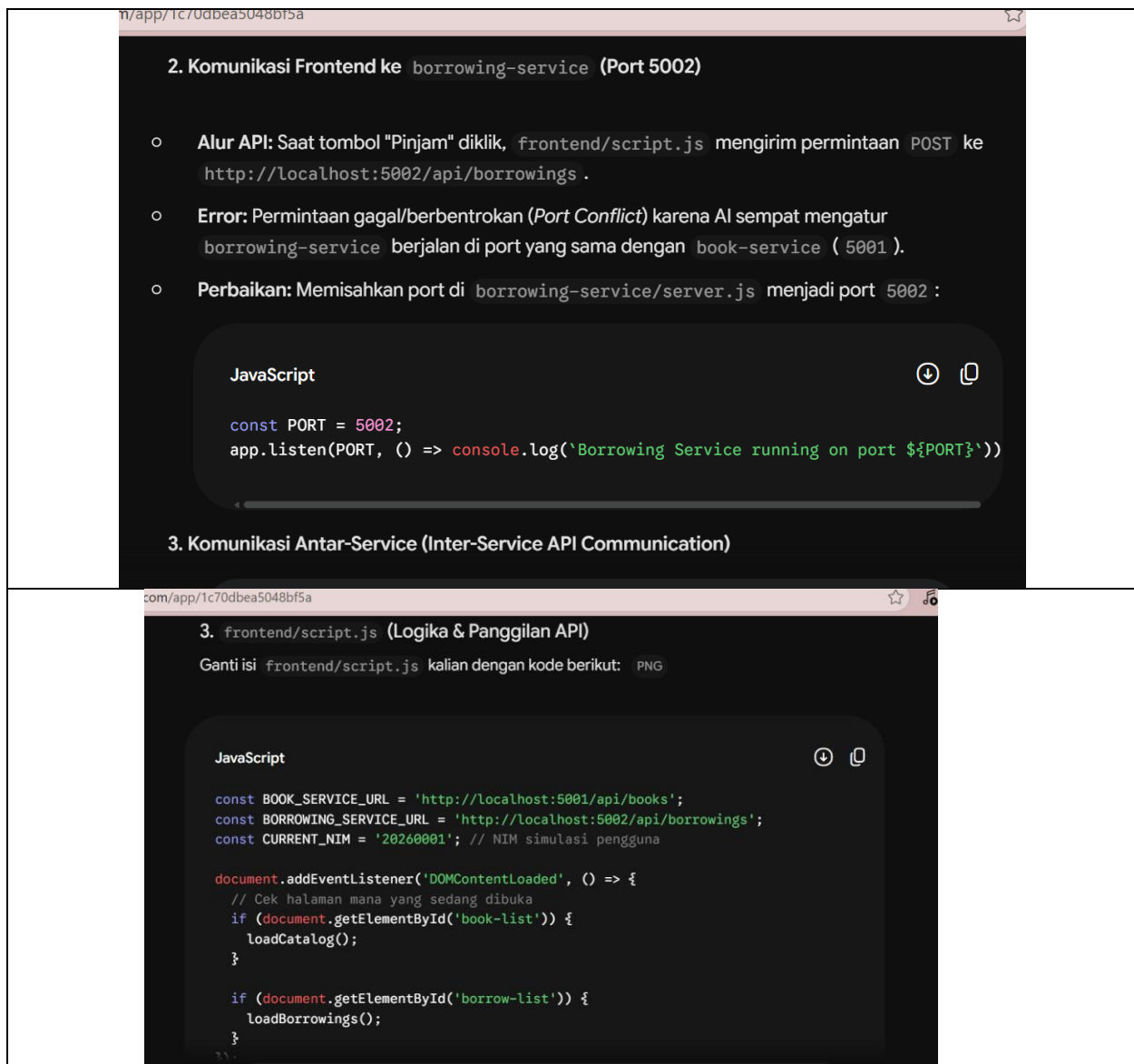
```
JavaScript

const cors = require('cors');
app.use(cors());
```

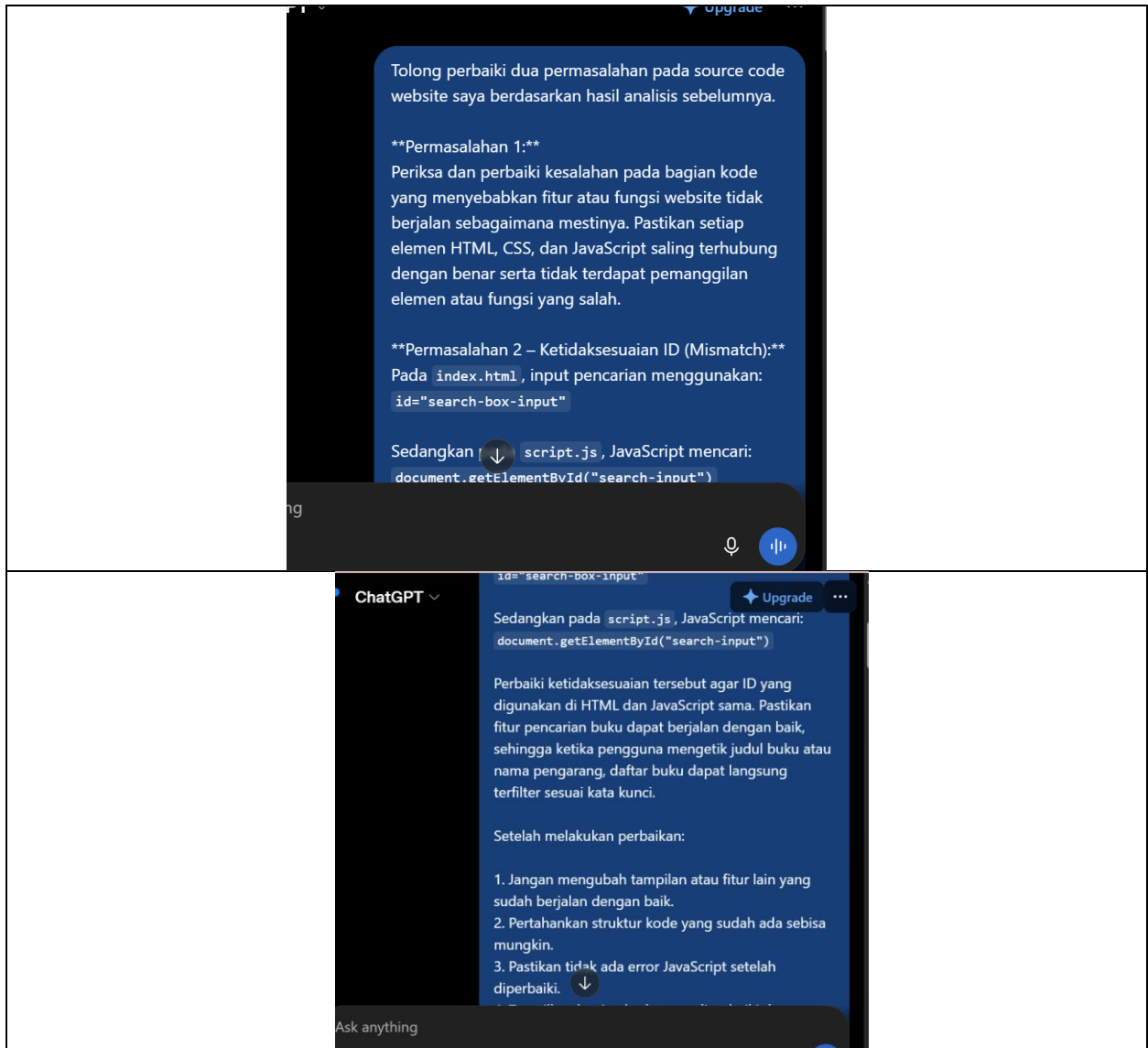
2. Komunikasi Frontend ke `borrowing-service` (Port 5002)

+ Minta Gemini Flash

Gemini adalah AI dan dapat membuat kesalahan.



Screenshot 5. Percakapan AI saat membantu memperbaiki error.



Setelah melakukan perbaikan:

★ Upgrade ...

1. Jangan mengubah tampilan atau fitur lain yang sudah berjalan dengan baik.
2. Pertahankan struktur kode yang sudah ada sebisa mungkin.
3. Pastikan tidak ada error JavaScript setelah diperbaiki.
4. Tampilkan bagian kode yang diperbaiki dan jelaskan secara singkat perubahan yang dilakukan.
5. Pastikan kedua permasalahan benar-benar sudah terselesaikan dan fitur website tetap berjalan normal.

Show less ^

- o Untuk Mengambil Data Buku (Ganti fetch/data lokal lama dengan API Book Service):

JavaScript



```
// Ambil data buku dari Book Service (Port 5001)
async function fetchBooks() {
  try {
    const response = await fetch('http://localhost:5001/api/books');
    const books = await response.json();
    renderBooks(books); // Panggil fungsi render UI kamu
  } catch (error) {
    console.error('Gagal mengambil data buku:', error);
  }
}
```

- o Untuk Peminjaman Buku (Ganti logika pinjam lama dengan API Borrowing Service):

o Untuk Peminjaman Buku (Ganti logika pinjam lama dengan API Borrowing Service):

JavaScript

```
// Kirim transaksi peminjaman ke Borrowing Service (Port 5002)
async function handleBorrow(bookId) {
  const nim = "20260001"; // Sesuaikan dengan NIM/User yang sedang login

  try {
    const response = await fetch('http://localhost:5002/api/borrowings', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ nim, bookId })
    });

    const result = await response.json();

    if (!response.ok) {
      alert(result.message); // Menampilkan pesan misal sudah capai batas 3 buku
      return;
    }
  }
}
```

Susunan letak folder dan file-filenya adalah sebagai berikut:

Plaintext

```
perpustakaan-microservice/ <-- FOLDER UTAMA PROYEK
├── frontend/               <-- FOLDER FRONTEND
│   ├── index.html          (file index.html lama)
│   ├── style.css           (file style.css lama)
│   └── script.js           (file script.js yang sudah diubah dipanggil API)
├── book-service/           <-- FOLDER BACKEND 1
│   ├── package.json        (dibuat pakai command: npm init -y)
│   └── server.js            (isi dengan kode Book Service)
├── borrowing-service/      <-- FOLDER BACKEND 2
│   ├── package.json        (dibuat pakai command: npm init -y)
│   └── server.js            (isi dengan kode Borrowing Service)
```

```

Plaintext
├── package.json
├── server.js
├── architecture/
│   └── architecture.png
└── README.md

```

(about pada command: npm init //
(isi dengan kode Borrowing Service)

<-- FOLDER DOKUMENTASI
(gambar diagram microservice)

<-- FILE PENJELASAN PROYEK

Cara Menjalankan & Membuka di VS Code

Buat Folder Utama

Buat folder baru di komputer dengan nama `perpustakaan-microservice`.

Buka di VS Code

Buka VS Code, lalu pilih **File > Open Folder** dan pilih folder `perpustakaan-microservice` tersebut.

3. Buat Sub-folder & File

Buat folder `frontend`, `book-service`, `borrowing-service`, dan `architecture` di dalam VS Code sesuai struktur di atas.

4. Inisialisasi Node.js di Masing-Masing Service

Buka Terminal di VS Code, lalu jalankan perintah berikut secara berurutan:

o Untuk Book Service:

```

Bash
cd book-service
npm init -y
npm install express cors

```

o Untuk Borrowing Service:

o Untuk Borrowing Service:

```

Bash
cd ../borrowing-service
npm init -y
npm install express cors

```

5. Cara Menjalankan Sistem Saat Testing:

Buka 2 Terminal di VS Code:

- o **Terminal 1:** Masuk ke folder `book-service` lalu jalankan `node server.js` (berjalan di port 5001).
- o **Terminal 2:** Masuk ke folder `borrowing-service` lalu jalankan `node server.js` (berjalan di port 5002).
- o **Frontend:** Buka file `frontend/index.html` langsung di browser atau gunakan ekstensi *Live Server* di VS Code.

Pada perbaikan kami juga menggunakan bantuan chat gpt.

9. HASIL IMPLEMENTASI

Setelah dilakukan pengembangan, sistem perpustakaan yang sebelumnya berada dalam satu folder telah dipisahkan menjadi dua service, yaitu:

1. Book Service : Digunakan untuk mengelola data buku dan mengecek apakah buku tersedia atau sedang dipinjam.
2. Borrowing Service : Digunakan untuk mengelola proses peminjaman dan pengembalian buku.

Kedua service dijalankan pada server dan port yang berbeda. Book Service menggunakan port 5001, sedangkan Borrowing Service menggunakan port **5002**. Kedua service dapat saling berkomunikasi menggunakan **API**. Contohnya, ketika pengguna ingin meminjam buku, Borrowing Service akan meminta informasi kepada Book Service untuk mengecek apakah buku tersebut masih tersedia. Selain itu, sistem juga memiliki aturan peminjaman, yaitu mahasiswa dapat meminjam maksimal **3 buku** dan lama peminjaman adalah **7 hari**. Hasil pengembangan kemudian diuji dengan menjalankan kedua server dan mencoba proses peminjaman serta pengembalian buku. Dari pengujian tersebut, komunikasi antar-service dan alur peminjaman dapat dijalankan sesuai dengan fungsi yang telah dibuat.

10. KESIMPULAN

Penggunaan AI Coding Tool membantu proses pengembangan proyek perpustakaan menjadi arsitektur dua service. AI digunakan untuk membantu perancangan, pembuatan kode, pemahaman API, serta pemecahan masalah. Meskipun demikian, hasil dari AI tetap diperiksa dan disesuaikan oleh pengembang agar sesuai dengan kebutuhan proyek dan dapat berjalan dengan baik.